Dynamic Storage Allocation with Reuse Penalties

Toni Böhnlein
Computing Systems Lab, Huawei Research Center Zurich
Zurich, Switzerland
toni.boehnlein@huawei.com

***Abstract*—Dynamic storage allocation (DSA) reduces memory use by allowing buffers with disjoint lifetimes to share addresses. On asynchronous accelerators, however, reuse may require synchronization between compute and memory-transfer pipelines and thereby reduce performance. Under a fixed capacity, the allocator must therefore choose which reuses are worth their synchronization cost.**

We introduce *DSA with reuse penalties* (DSA-RP) to co-optimize memory placement and reuse-induced synchronization for a given logical schedule. Each selected buffer pair incurs a penalty when its address ranges overlap. The allocator minimizes the total penalty while respecting lifetime conflicts and capacity. We prove that DSA-RP is NP-hard even for unit-size buffers with disjoint lifetimes. We develop exact algorithms and a practical greedy heuristic, and give polynomial-time algorithms for restricted unit-size pipelines.

We implement the heuristic in an ML compiler and evaluate it on 36 structurally selected configurations of nine inference-kernel families from DeepSeek and Qwen implementations. It achieves latency reductions reaching approximately 27% over a conventional first-fit allocator that ignores synchronization costs and 15% over a baseline based on Cypress [7].

I. Introduction

AI accelerator compilers assign intermediate tensors to an explicitly managed scratchpad of fixed capacity. Classical *dynamic storage allocation* (DSA) seeks a minimum-footprint placement of buffers with given sizes and lifetimes: buffers whose lifetimes overlap must occupy disjoint address ranges. Buffers with disjoint lifetimes may share addresses, and this reuse carries no cost of its own in the formulation.

That reuse can nevertheless restrict execution. Compute and DMA engines operate asynchronously, so the logical order used to derive lifetimes does not necessarily order accesses across engines. If an access to an old buffer and the first write to a new buffer are unordered, overlapping their addresses requires an additional dependency. Enforcing it can delay prefetching or serialize work that could otherwise overlap. The effect depends on the available slack: an added dependency may leave execution time unchanged or delay completion. A smaller footprint is therefore not necessarily a better-performing placement at the capacity available.

Figures 1 and 2 illustrate this distinction with the computation $(a \cdot b)^2 + c$ on a vector unit and a DMA engine, with all tensors of size s. In-place execution folds d, e, and f into a's storage, so the chain $a \to d \to e \to f$ occupies one slot throughout. At capacity $2s$, the remaining operands b and c must share the second slot. Their logical lifetimes are disjoint, but the multiply that last accesses b and the DMA operation that writes c are unordered; sharing the slot therefore requires a synchronization. At capacity $3s$, c can occupy a separate slot, avoiding this synchronization and allowing its load to overlap the multiply. The placement does not itself move the load: it removes the reuse-induced constraint that would prevent such overlap.

Capacity is therefore a budget for avoiding reuse-induced dependencies. Keeping every potentially costly pair apart may exceed that budget; minimizing footprint may instead introduce dependencies that the available capacity would allow us to avoid. The allocator must choose which reuses are worth their synchronization cost.

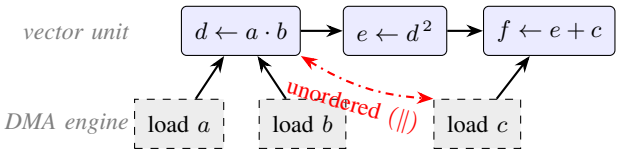


Fig. 1. The computation $(a \cdot b)^2 + c$ on two engines. Each lane executes in order; solid arrows are data dependencies. The multiply and the load of c are unordered. Reusing b's addresses for c therefore requires a cross-engine synchronization.

The memory–parallelism trade-off already appears in allocation policies. A stream-ordered allocator may insert a dependency before reusing memory freed on another stream; this can serialize otherwise concurrent work [1]. For compiler-managed shared memory, Cypress adds optional interference edges and removes them until an allocation fits the user-specified budget [7]. We instead make estimated reuse cost an explicit allocation objective, so that the allocator can account for synchronization consequences while choosing addresses.

A. Our Contributions

The overview below includes results from the full version of this paper. Its theorem, lemma, proposition, corollary, and section references use that version's numbering; some of these results are omitted from this short version. The remaining sections here develop the placement foundation, compiler penalty construction, and experimental evaluation.

We introduce *DSA with reuse penalties* (DSA-RP) to co-optimize memory placement and reuse-induced synchronization. The schedule is an input: operation-to-pipeline assignments, per-pipeline operation orders, and the logical order used to derive buffer lifetimes remain fixed. Placement can nevertheless change execution overlap by determining which

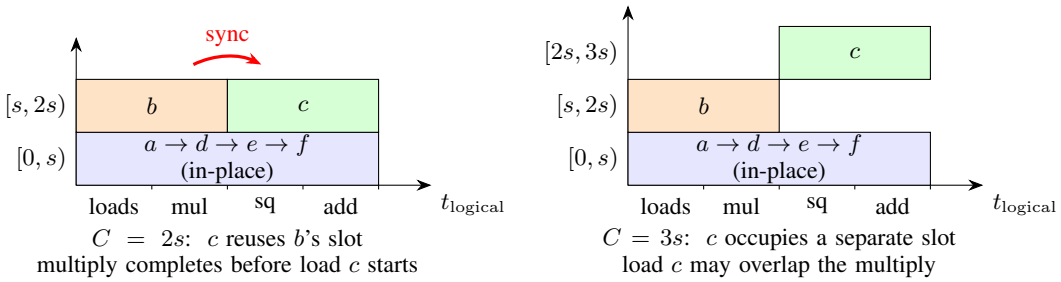


Fig. 2. Two address placements with identical logical lifetimes (horizontal axis; addresses vertical). At capacity $2s$, sharing b's slot requires the multiply to complete before the load of c begins. At capacity $3s$, a separate slot removes this reuse-induced dependency, permitting the load of c to overlap the multiply.

additional dependencies storage reuse requires. Lifetime conflicts and capacity are hard constraints. Weighted *soft edges* price selected overlaps between lifetime-disjoint buffers: an edge is activated exactly when its two buffers' address ranges overlap. The allocator minimizes the total activated weight at the given capacity. Thus the model distinguishes which buffers may legally share storage from which legal reuses are worth accepting.

Our main contributions are:

1) **Model and complexity.** We characterize the capacity–synchronization frontier and establish NP-hardness and approximation barriers even for unit-size buffers with disjoint lifetimes.
2) **Structural foundations and algorithms.** We extend the support and ordering principles of canonical DSA search [6] to retain penalty-optimal placements, yielding exact algorithms, greedy construction, and local search.
3) **Compiler integration and performance.** We implement structural penalty recognition and canonical-greedy allocation in the PyPTO ML compiler [2]. On structurally selected LLM inference-kernel configurations, latency reductions reach approximately 27% over a first-fit allocator that ignores synchronization costs and 15% over a Cypress-based baseline [7], at unchanged memory capacity.

Varying the capacity reveals how much memory is needed to avoid a given reuse cost. The minimum penalty forms a nonincreasing staircase, the *capacity–synchronization frontier* (Section 3). Its left endpoint is the classical DSA optimum, the least capacity at which any placement fits. Its zero-penalty threshold is the least capacity at which all soft pairs can be separated simultaneously. Between these thresholds, additional memory can avoid penalties that a minimum-footprint placement incurs.

This choice remains difficult after the usual packing complications are removed. With unit buffers and disjoint lifetimes, capacity two gives two address slots: maximizing avoided penalty is MAXCUT, whereas minimizing incurred penalty is MINUNCUT. For every fixed $C \geq 3$, zero penalty is possible exactly when the soft graph is C-colorable. Because any finite multiplicative approximation must recognize a zero optimum, no deterministic polynomial-time algorithm can provide such a guarantee at these capacities unless $\mathsf{P} = \mathsf{NP}$ (Section 3). The obstacle is therefore intrinsic to choosing reuses, not only to packing unequal buffers.

The penalty objective also changes what a complete search must represent. Canonical DSA search can retain a minimum-footprint solution by considering *grounded* placements: each buffer starts at zero or immediately above a buffer with which it has a hard conflict [6]. For DSA-RP, this restriction can exclude every minimum-penalty placement. We exhibit instances on which every hard-grounded placement pays the entire penalty weight although a zero-penalty placement fits. No search order or rescoring rule can recover placements absent from the representation (Section 4).

Our central structural result repairs this loss by preserving the separations that make a placement valuable. Any feasible placement can be lowered to a grounded one without increasing its height or activating a previously separated soft edge (Lemma 3.2). Each nonzero offset then lies immediately above a hard-conflicting buffer or a separated soft partner. Orienting these constraints by address order gives least offsets through longest paths, without requiring an interval graph (Theorem 4.1). We can therefore either *promote* selected soft edges and solve the grounded feasibility problem, or admit soft partners directly into canonical enumeration while retaining its offset ordering (Theorems 4.3–4.6). This extends the support and ordering principles to penalty optimization, but does not automatically preserve footprint-based pruning rules.

Promotion converts certified infeasibility into lower bounds on reuse cost. At a fixed capacity where the hard constraints fit, a *core* is a minimal set of soft pairs that cannot all be separated. Every fitting placement must activate an edge from every core. Conversely, meeting all cores permits a fitting placement whose activated edges lie within the chosen set. Minimum penalty therefore equals the minimum weight of a hitting set of the cores (Proposition 3.4). An implicit-hitting-set algorithm [9], [10] discovers cores as needed, alternating between allowed activations and feasibility tests. Exact master and feasibility solves certify an optimum; joint canonical branch-and-bound instead searches placements directly (Section 6.2). Cores remain valid as capacity decreases, allowing their reuse along the frontier.

The decomposition does not require independent pair charges. One synchronization action may enforce the ordering

required by several reuses. We extend the placement–hitting-set identity to any inclusion-monotone activation cost on a fixed geometric instance (Theorem 3.6). With exact subproblem solutions, the same algorithm can therefore optimize the cost of shared synchronization actions rather than charging each reuse independently.

Temporal structure offers a different route to exact optimization. Consider unit buffers grouped into phases, with pairwise-overlapping lifetimes within each phase, disjoint lifetimes across phases, and penalties only between adjacent phases. Each memory slot follows a path through the phases, with reuse costs charged on its transitions. This gives a minimum-cost-flow algorithm for both an optimal placement and the entire frontier (Theorem 4.9). For singleton phases, two alternating slots avoid every penalty: double buffering is optimal in this class. Allowing penalties between phases two steps apart makes the zero-penalty decision NP-complete already with three full phases and unit weights, when capacity is part of the input (Proposition 4.11).

Other algorithms exploit a small parameter rather than a phase order. For unit buffers at capacity two, each feasible hard-conflict component has two slot assignments, differing by a flip. Optimizing these flips reduces the problem to weighted MINUNCUT, giving a randomized $O(\sqrt{\log n})$ approximation for n buffers. With positive integer weights, we also obtain exact algorithms parameterized by the numerical optimum, even when weights are binary-encoded (Proposition 4.13 and Theorem 4.14). More generally, a supplied tree decomposition of the combined hard-and-soft graph confines constraints to small bags. For unit buffers, recording which bag vertices share a slot, rather than recording slot names, gives fixed-parameter tractability in treewidth alone: capacity need not enter the state count (Theorem 4.17).

Beyond exact optimization, the attainable guarantee depends on what may be relaxed. For unit buffers with no hard conflicts, MAX-C-CUT approximation gives an additive penalty bound at the original capacity (Theorem 4.19). On unit instances admitting zero penalty, retaining zero penalty with more slots is approximate coloring of the combined graph; coloring algorithms and their limitations therefore transfer (Proposition 4.15 and Corollary 4.16).

For unequal sizes, we obtain a capacity-augmented guarantee under *scale separation*: every soft edge joins buffers of size at least δC (Section 5). Small buffers contribute no penalties and occupy a separate band using a classical packing bound [8]. The large-buffer problem is discretized by rounding sizes and reconstructing offsets while preserving the required separations. A sweep must retain a buffer's offset even after its lifetime ends if a future soft partner can still incur a penalty. We bound this history through the *birth span*: the number of large-buffer births after the earlier endpoint's lifetime ends and up to the later endpoint's birth. For fixed $\varepsilon \in (0, 1]$, sufficiently small fixed δ, and fixed birth span, a polynomial-time algorithm attains penalty at most the optimum for capacity C, using at most $(2 + \varepsilon)C$ space (Theorem 5.8). A supplied bounded-width decomposition of the large combined graph gives an alternative with the same guarantee (Corollary 4.18). Scale separation alone does not suffice: it still admits the coloring obstruction.

The same representations yield practical heuristics for general instances (Section 6). Canonical greedy compares capacity-feasible candidates from the extended support menu by their incremental penalty. Promote-and-repair starts by forbidding every penalized reuse, then relaxes separations and repacks. Local search changes the buffer-placement order and the promoted set, not the operation schedule. For every feasible placement, we prove that an order-and-promotion pair decodes to a placement of no greater height or penalty (Lemma 6.1). The representation retains an optimum, although local-search moves need not reach it.

We compare nine general heuristics on eleven MiniMalloc-derived instances with variable buffer sizes and unit penalties (Section 8.2). Canonical greedy matches promote-and-repair's penalty on every instance. Both reduce summed penalty by 7.2% relative to geometry-only first fit, but canonical greedy takes 5.8–56.7 ms of host-planning time, compared with 0.96–21.34 s for promote-and-repair. On this corpus, direct penalty-aware construction thus matches the quality of repeated relaxation and repacking at much lower planning cost. Local search reduces the first-fit total by 16.9% at greater planning cost.

To connect allocation to execution, we derive reuse requirements from access ordering (Section 7). A buffer may have several mutually unordered final accesses, so the last operation in the logical schedule is not enough to characterize its reuse requirements. Under the stated access-ordering and allocation-granularity assumptions, reuse needs no added dependency exactly when the existing order guarantees that every maximal access to the old buffer finishes before the new buffer's first write begins. For a scheduled-program model with k streams, constructing a reachability index takes $O(k(V + E))$ time for V access events and E dependency edges. A buffer-pair test then uses at most k constant-time ordering queries. We implement a structural recognizer in PyPTO [2], a compiler for machine-learning kernels. It approximates this criterion before synchronization insertion using full-allocation read/write effects and resource information, merging witnesses into one unit-weight edge per recognized cross-resource buffer pair. The weights are uniform, but the edge set distinguishes selected reuse hazards from physical aliasing alone. Neither operation-duration estimates nor synchronization simulation are needed; the downstream pass enforces the ordering required by the chosen placement.

Finally, we evaluate the recognizer and canonical-greedy allocator against three alternative placement policies on 36 structurally selected configurations from nine inference-kernel families in DeepSeek and Qwen implementations (Section 8.3). Every comparison fixes the logical schedule, kernel parameters, and memory capacity. Latency reductions reach approximately 27% over first fit and 15% over a baseline based on Cypress [7]; the headline gains occur on different configurations and replicate on two devices. These results show that even unit penalties derived from access structure

can guide placements that improve kernel performance at the same memory capacity.

II. Placement Choices at Fixed Capacity

Buffer i has positive integer size s_i, a nonempty half-open logical lifetime, and address range $J_i(p) = [p_i, p_i + s_i)$ at offset $p_i \geq 0$. Hard pairs T overlap in lifetime and require disjoint ranges. Soft pairs P are lifetime-disjoint, with reuse costs $w_{ij} > 0$. At capacity C, we minimize

$$\text{cost}(p) = \sum_{\{i,j\} \in P} w_{ij} \mathbf{1}[J_i(p) \cap J_j(p) \neq \emptyset]$$

subject to hard separation and $\max_i(p_i + s_i) \leq C$. The operation schedule stays fixed; pair costs need not equal physical synchronization counts or latency.

Canonical footprint search uses supports from hard-conflicting buffers [6]. Two lifetime-disjoint unit buffers joined only by a unit soft edge expose its limitation: hard-only grounding puts both at zero and pays one, whereas offsets zero and one avoid cost at capacity two. Rescoring cannot recover a choice absent from the representation. Adding soft partners repairs that loss, including under fixed alignment.

Proposition II.1. *For any feasible aligned placement p with fixed integer alignments $a_j > 0$, there is an aligned placement $q \leq p$ preserving every hard and soft separation of p. Each nonzero q_j is an aligned end of a hard or soft partner. Offset-ordered enumeration using those supports retains a penalty optimum.*

Orient hard pairs and separated soft pairs by their address order in p. Positive sizes make the orientation acyclic. In topological order, set

$$q_j = a_j \left\lceil \frac{\max(\{0\} \cup \{q_i + s_i : i \to j\})}{a_j} \right\rceil .$$

Every predecessor end stays below the aligned offset p_j, so induction gives $q_j \leq p_j$ while preserving the oriented separations. A predecessor attaining a nonzero maximum supplies a support strictly below q_j; offset order places it first. Without alignment, omit the rounding to obtain the unaligned result. Reserved ranges require separate justification, and completeness of enumeration does not make greedy traversal optimal.

III. Constructing and Using Penalty Edges

A. Compiler inputs and access frontiers

PyPTO [2] fixes allocation identities and the logical operation schedule, exporting sizes, arenas, alignments, lifetimes, and hard constraints. The recognizer constructs a unit-weight catalog P_{impl} for the objective of Section II. After allocation, PTOAS's unchanged synchronization pass [5] enforces required ordering, including hazards omitted from that catalog.

Source order alone cannot identify every reuse requirement. If final reads r_1, r_2 on different engines are unordered, ordering r_1 before the next buffer's first write w leaves $r_2 \to w$ to enforce. In a finite access-order model with one initial write preceding all later accesses and conservative whole-allocation reuse, no added order is needed exactly when every maximal old access is guaranteed to finish before that write starts. The recognizer approximates this criterion with static frontiers.

Registered operation effects identify reads and writes; outputs add writes not already represented. Transfer routes and memory spaces classify DMA, vector, and matrix resources. Endpoints record byte ranges and branch/loop provenance; unsupported accesses mark incomplete allocation summaries. The terminal frontier retains the latest issue endpoint per resource, arena, control context, and range description, then removes endpoints preceding another retained endpoint under an ordering proxy. That proxy uses resource issue order and cross-resource SSA reachability on compatible paths; enclosing statements in the deepest common region represent cross-region dependencies. The initial frontier retains endpoints with no predecessor under the same proxy. Initial reads or accesses lacking an initial anchor trigger fallback and exclude that target from scoring.

B. From handoff witnesses to pair costs

For lifetime-disjoint allocations in one arena, the recognizer combines the earlier terminal and later initial frontiers. Compatible endpoints in logical order witness write-after-read (WAR) or write-after-write (WAW) handoffs when an old read or write precedes a new write. The filters below select the witnesses contributing to the catalog.

Information	Treatment in the scored catalog
Coverage	Exclude partial/unknown ranges, incomplete summaries, and conservative initial anchors.
Resources	Retain cross-resource witnesses; exclude same-resource and pipeline-serialization candidates.
Control/aliases	Require compatible contexts; exclude unresolved same-operation alias contracts.
Ordering	SSA-order evidence alone does not exclude a witness.
Aggregation	One unit-weight edge per unordered allocation pair.

SSA reachability helps select frontiers but need not guarantee asynchronous access completion. A dependency path therefore does not itself discard a handoff. Frontier reduction and filtering can also omit requirements: the catalog is neither proved a subset nor a superset of exact hazards. It ignores frequency, slack, and shared synchronization actions. Its score guides placement; it certifies neither safety nor synchronization count.

C. Static loops and branch compatibility

The recognizer visits loop bodies without unrolling unknown iteration counts. Supported shared-loop contexts admit forward handoffs within an iteration and reverse distance-one handoffs. In the repeated body

```
load A using DMA; compute using A;
load B using DMA; compute using B,
```

when logical lifetimes permit reuse, reads of $A(t)$ before writes to $B(t)$ give forward WAR witnesses. If another iteration executes, reads of $B(t)$ before writes to $A(t+1)$ give reverse witnesses. Both merge into one unit edge $\{A, B\}$, without predicting occurrence counts or delay.

TABLE I

REPRESENTATIVE GAINS AND A LIMITATION OF THE STRUCTURAL SCORE. AT FIXED CAPACITY AND SCHEDULE, RP SUBSTANTIALLY REDUCES LATENCY IN TWO CONFIGURATIONS BUT IS SLIGHTLY SLOWER ON O-LORA DESPITE A LOWER SCORE. SCORES COUNT RECOGNIZED BUFFER PAIRS SHARING ADDRESSES, NOT SYNCHRONIZATION INSTRUCTIONS. DEVICE COLUMNS GIVE PAIRED LATENCY CHANGES (%) WITH 95% INTERVALS; NEGATIVE FAVORS RP. O-LORA'S SLOWDOWN IS BELOW THE 2% CONFIRMATION THRESHOLD. PARAMETERS DESCRIBE TILES, NOT FULL TENSOR DIMENSIONS.

Configuration	Capacity (bytes)	Structural score			Comparison	Latency change (%)	
		GFF	CYP	RP		Device A	Device B
Index-QR						-27.07	-27.43
$T = 8, Q_{\text{out}} = 1024$	112,656	5	1	0	RP/GFF	$[-28.01, -25.93]$	$[-28.05, -26.79]$
MTP dequant.						-16.82	-14.41
$T = 8, O = 1024$	188,416	9	3	0	RP/CYP	$[-19.14, -14.76]$	$[-16.02, -12.68]$
O-LoRA						$+1.40$	$+1.13$
$K = 256, T = 8$	32,864	18	20	7	RP/CYP	$[0.67, 2.21]$	$[0.39, 1.87]$

Opposite conditional arms exclude same-iteration handoffs. Across a shared loop, an internal branch may change arms between iterations; an external branch remains mutually exclusive. This checks structural possibilities, not general path feasibility or arbitrary-distance/index-dependent memory dependences. Omitted hazards remain the backend's responsibility.

D. Greedy address selection

For buffer j, canonical greedy considers zero and placed hard-or-soft partner ends, rounded to $a_j\lceil(p_i + s_i)/a_j\rceil$; reserved-range ends are included when present. It deduplicates candidates, checks hard conflicts, reserved ranges, and capacity, then minimizes newly activated weight against placed partners, breaking ties by offset. Buffer orders and restarts provide alternative constructions without rescheduling operations. Failed fit does not prove infeasibility.

In Section II's example, after one buffer is placed at zero, its partner has candidates zero and one, costing one and zero. Greedy selects the useful separation. This method is CG in the solver study and RP with P_{impl} in the device study.

IV. EVALUATION

A. Policies and controlled conditions

We compare RP with geometry-only first fit (GFF), footprint-oriented order/restart search (GCG), and a Cypress-style control (CYP) on 36 configurations from nine DeepSeek/Qwen inference-kernel families in PyPTO-Lib [3]. CYP relaxes optional separations under six weight-blind orders, selecting by feasibility, physical alias count, relaxed pairs, and footprint; it does not reproduce the complete Cypress compiler [7]. RP and GCG use seed zero and eight restarts. Identical maps share physical measurement endpoints.

Compiler versions, pre-synchronization operations, schedule, parameters, alignments, constraints, and capacity are fixed across policies. Returned placements are validated and checked against emitted addresses. Of the retained vector-arena budgets, 18 use the GFF footprint; 11 use that footprint plus one quarter of the remaining headroom to native capacity, three use the footprint plus half that headroom, and four use the native 184 KiB. Fractional headroom is rounded to integer bytes. The cohort requires reuse, all policies fitting, distinct GFF/CYP/RP maps, and lower RP cost than CYP. Equal native-problem fingerprints favor the largest CYP–RP score gap. Selection does not consult latency; without prefilter counts, it does not estimate opportunity prevalence.

Two devices of the same accelerator model use balanced paired blocks; hardware identity is withheld under disclosure restrictions. We measure each task's device-domain execution window, excluding compilation, Python execution, and CPU reference computation. Each endpoint/block has three warmups and ten measured rounds. A uses one Williams cycle; effects of at least 1% with 95% intervals excluding zero advance to two cycles on B with reversed endpoint order. All 36 configurations run on A, 27 on B. Effects are geometric means of paired within-block ratios; intervals use 5,000 whole-block bootstrap resamples per device. Confirmation requires at least 2% magnitude, intervals excluding zero, and matching direction on both devices, without simultaneous error control.

B. Performance gains and their limits

RP has 19 confirmed improvements over GFF and 18 over CYP. Table I shows large gains on different configurations and an adverse case. Index-QR reduces GFF's score from five to zero and improves latency by about 27% on both devices. MTP reduces CYP's score from three to zero at native capacity, improving latency by 16.82% and 14.41%. Hidden normalization and bias show repeated gains; KV commit/bias, split pre/post, and sampling have no confirmed differences. These are selected-configuration results, not workload-wide averages.

All 49 confirmed pairs among 111 physical comparisons agree with score order. Removing the 2% cutoff exposes O-LoRA reversals against both GFF and CYP: lower RP cost accompanies slightly worse latency. Four same-binary control blocks on A produce apparent effects of $+2.52\%$ and -2.84%. These are not a calibrated noise bound or controls under the full two-device protocol, and limit interpretation of borderline gains. Compiler overhead and traces identifying the responsible waits remain unmeasured. The comparisons assess

complete allocation policies, without isolating recognition from support selection.

C. Placement quality and planning cost

On eleven MiniMalloc-derived variable-size instances (154–454 buffers, 1,928–9,890 unit-weight pairs from deterministic four-stream accesses), capacity is the GFF peak. All nine general heuristics fit. Stochastic runs use seeds 0–2, 2,000 evaluations, four restarts, and stagnation limit 250; deterministic methods run once [4]. CG matches promote–repair's best validated cost on every instance: the sum falls from GFF's 6,698 to 6,217 (7.2%), entirely through F (616 to 135). Median host times are 5.8–56.7 ms versus 0.96–21.34 s for repair, which repeatedly relaxes soft separations and repacks. Local search lowers GFF's total by 16.9%, with different initialization as well as moves. No general exact engine certifies an optimum within 60 seconds and two million nodes; specialized engines are all out of class. These timings start with a constructed penalty graph and establish neither heuristic optimality gaps nor total compiler overhead.

V. Conclusion

Pricing selected reuse hazards changes scratchpad allocation at fixed capacity and can improve measured kernel latency. Extended supports supply the needed placement choices; a structural recognizer supplies a practical score while the backend enforces synchronization. For virtual-to-physical lowering, memory feasibility and reuse ordering therefore belong in the same allocation decision. Compiler-overhead measurements, synchronization traces, and broader cohorts are the next steps; joint scheduling must reconstruct lifetimes and costs.

References

[1] NVIDIA. Stream-Ordered Memory Allocator. In *CUDA Programming Guide*, Sections 4.3.3 and 4.3.4.4. https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/stream-ordered-memory-allocation.html. Accessed September 9, 2026.

[2] *PyPTO*. Compiler software, research snapshot used in this evaluation, 2026. Commit `3eabcfd22894151cbda6c752dcb300708a34f28d`. https://github.com/tonibohnlein/pypto.

[3] *PyPTO-Lib*. Inference-kernel library, research snapshot used in this evaluation, 2026. Commit `83e19f6b06eb2125eb14c06232358f342941c8d5`. https://github.com/tonibohnlein/pypto-lib.

[4] *DSA Solver*. Dynamic storage allocation and reuse-penalty solvers, evaluation snapshot, 2026. Commit `60ac39b02b008dc3907b876ad47a1cc342aa01fa`. https://github.com/tonibohnlein/dsa-solver.

[5] *PTOAS*. PTO assembler, release v0.57, 2026. https://github.com/hw-native-sys/PTOAS/releases/tag/v0.57.

[6] M. D. Moffitt. MiniMalloc: A lightweight memory allocator for hardware-accelerated machine learning. In *ASPLOS*, 2023.

[7] R. Yadav, M. Garland, A. Aiken, and M. Bauer. Task-based tensor computations on modern GPUs. *Proc. ACM Program. Lang.* 9(PLDI), Article 163, 2025.

[8] A. L. Buchsbaum, H. Karloff, C. Kenyon, N. Reingold, and M. Thorup. OPT versus LOAD in dynamic storage allocation. *SIAM J. Computing* 33(3):632–646, 2004.

[9] K. Chandrasekaran, R. Karp, E. Moreno-Centeno, and S. Vempala. Algorithms for implicit hitting set problems. In *SODA*, 2011.

[10] E. Moreno-Centeno and R. M. Karp. The implicit hitting set approach to solve combinatorial optimization problems with an application to multigenome alignment. *Operations Research* 61(2):453–468, 2013.